VIT®
BHOPAL
www.vitbhopal.ac.in

CSE 2006 - Programming in Java

Course Type: LP Credits: 3

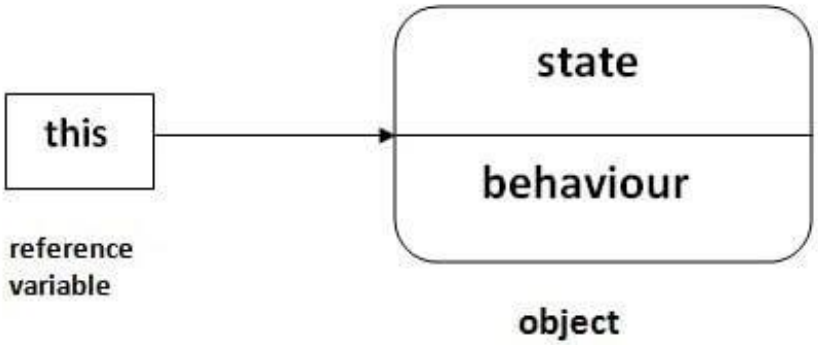
Prepared by
Dr Komarasamy G
Associate Professor (Grade-2)
School of Computing Science and Engineering
VIT Bhopal University

Unit-2 Java Object-oriented Programming

- **Java OOP (Basics)**
- Java Class and Objects, Java Methods, Java Constructor, Java Strings, Java Access Modifiers, **Java this keyword, Java final keyword, Java Recursion, Java instance of Operator, Java Single Class and Anonymous Class, Java enum Class**
- **Java OOP (Inheritance & Polymorphism)**
- Java Inheritance, Java Method Overriding, Java super Keyword, Abstract Class & Method, Java Interfaces, Java Polymorphism (overloading & overriding), Java Encapsulation
- **Java OOP (Other types of classes)**
- Nested & Inner Class, Java Static Class, Java Anonymous Class, Java Singleton, Java enum Class, Java enum Constructor, Java enum String, Java Reflection

this keyword in Java

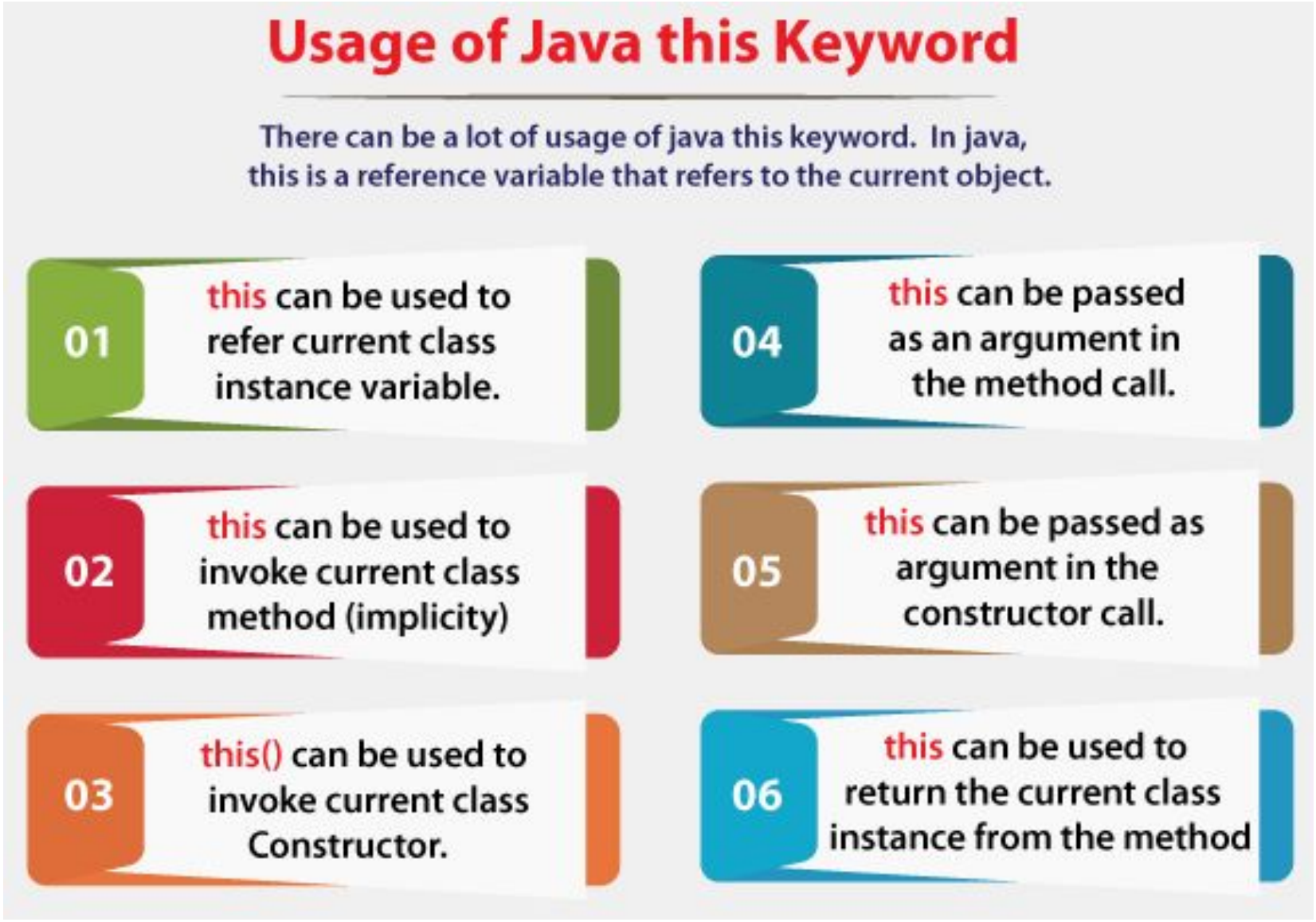
- There can be a lot of usage of **Java this keyword**. In Java, this is a **reference variable** that refers to the current object.



- **Usage of Java this keyword**
  - **Here is given the 6 usage of java this keyword.**

1. **this can be used to refer current class instance variable.**
2. **this can be used to invoke current class method (implicitly)**
3. **this() can be used to invoke current class constructor.**
4. **this can be passed as an argument in the method call.**
5. **this can be passed as argument in the constructor call.**
6. **this can be used to return the current class instance from the method.**

this keyword in Java



this keyword in Java

- **1) this: to refer current class instance variable**
- The this keyword can be used to refer **current class instance variable.**
- If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

this keyword in Java

- **Understanding the problem without this keyword**
- Let's understand the problem if we don't use this keyword by the example given below:

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,
                float fee)
{
rollno=rollno;
name=name;
fee=fee;
}
void display()
{System.out.println(rollno+" "+name+" "+fee);}
}
```

```
class TestThis1{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

0 null 0.0

0 null 0.0

In the above example, parameters (formal arguments) and instance variables are same. So, we are using this keyword to distinguish local variable and instance variable.

this keyword in Java

- **Solution of the above problem by this keyword**

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,
            float fee)
{
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}
```

```
class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

111 ankit 5000.0

112 sumit 6000.0

If local variables(formal arguments) and instance variables are different, there is no need to use this keyword like in the following program

this keyword in Java

- **Program where this keyword is not required**

```
class Student{
int rollno;
String name;
float fee;
Student(int r,String n,float f){
rollno=r;
name=n;
fee=f;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}
```

```
class TestThis3{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

Output:

111 ankit 5000.0

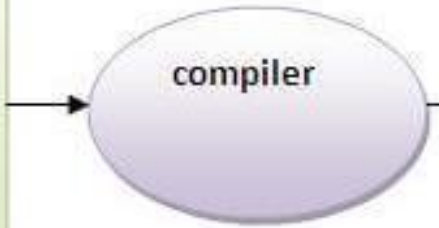
112 sumit 6000.0

It is better approach to use meaningful names for variables. So we use same name for instance variables and parameters in real time, and always use this keyword.

this keyword in Java

- **2) this: to invoke current class method**
- You may invoke the method of the current class by using the this keyword. If you don't use the this keyword, compiler automatically adds this keyword while invoking the method. Let's see the example

```
class A{

void m(){}

void n(){
m();
}

public static void main(String args[]){

new A().n();

}}
```




```
class A{

void m(){}

void n(){
this.m();
}

public static void main(String args[]){

new A().n();

}}
```

this keyword in Java

```
class A{
void m(){System.out.println("hello m");}
void n(){
System.out.println("hello n");
//m();//same as this.m()
this.m();
}
}
class TestThis4{
public static void main(String args[]){
A a=new A();
a.n();
}}
```

Output:

hello n

hello m

this keyword in Java

- **3) this() : to invoke current class constructor**
- The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.
- **Calling default constructor from parameterized constructor:**

```
class A{
A(){System.out.println("hello a");}
A(int x){
this();
System.out.println(x);
}
}
class TestThis5{
public static void main(String args[]){
A a=new A(10);
}}
```

Output:

```
hello a
10
```

this keyword in Java

- **Calling parameterized constructor from default constructor:**

```
class A{
A(){
this(5);
System.out.println("hello a");
}
A(int x){
System.out.println(x);
}
}
class TestThis6{
public static void main(String args[]){
A a=new A();
}}
```

Output:

5

hello a

this keyword in Java

- **Real usage of this() constructor call**
- The this() constructor call should be used to reuse the constructor from the constructor. It maintains the chain between the constructors i.e. it is used for constructor chaining. Let's see the example given below that displays the actual use of this keyword.

- **More details:**

  https://www.javatpoint.com/this-keyword

this keyword in Java

- **Real usage of this() constructor call**

```
class Student{
int rollno;
String name,course;
float fee;
Student(int rollno,String name,
            String course){
this.rollno=rollno;
this.name=name;
this.course=course;
}
Student(int rollno,String name,String course,float fee){
this(rollno,name,course);//reusing constructor
this.fee=fee;
}
void display(){System.out.println(rollno+  +name+  +course+  +fee);}}
```

```
class TestThis7{
public static void main(String args[]){
Student s1=new Student(111,"ankit","java");
Student s2=new Student(112,"sumit","java",6000f);
s1.display();
s2.display();
}}
```

Output:

111 ankit java 0.0

112 sumit java 6000.0

Rule: Call to this() must be the first statement in constructor.

Java final keyword

- The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:
- **variable**
- **method**
- **class**
- The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

1) Java final variable

- If you make any variable as final, you cannot change the value of final variable(It will be constant).
- **Example of final variable**
- There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

Java final keyword

```
class Bike9{
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
 Bike9 obj=new  Bike9();
 obj.run();
 }
}//end of class
```

Output: Compile Time Error

Java final keyword

- **2) Java final method**
- If you make any method as final, you cannot override it.
- **Example of final method**

```
class Bike{
  final void run(){System.out.println("running");}
}

class Honda extends Bike{
   void run(){System.out.println("running safely with 100kmph");}

   public static void main(String args[]){
   Honda honda= new Honda();
   honda.run();
   }
}
```

Output: Compile Time Error

Java final keyword

- **3) Java final class**
- If you make any class as final, you cannot extend it.
- **Example of final class**

```
final class Bike{}

class Honda1 extends Bike{
  void run(){System.out.println("running safely with 100kmph");}

  public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

Output: Compile Time Error

More details: https://www.javatpoint.com/final-keyword

Applying final keyword in Java

Q) What is blank or uninitialized final variable?

A final variable that is not initialized at the time of declaration is known as blank final variable.

If you want to create a variable that is initialized at the time of creating object and once initialized may not be changed, it is useful. For example PAN CARD number of an employee.

It can be initialized only in constructor.

Example of blank final variable

```
class Student
{
int id;
String name;
final String PAN_CARD_NUMBER;
...
}
```

Applying final keyword in Java

Que) Can we initialize blank final variable?

Yes, but only in constructor. For example:

```
class Bike10{
  final int speedlimit;//blank final variable

  Bike10(){
  speedlimit=70;
  System.out.println(speedlimit);
  }

  public static void main(String args[]){
    new Bike10();
  }
}
```

Output:
70

Applying final keyword in Java

static blank final variable

A static final variable that is not initialized at the time of declaration is known as static blank final variable. It can be initialized only in static block.

Example of static blank final variable

```
class A
{
  static final int data;    //static blank final variable
  static
      { data=50;}
  public static void main(String args[])
{
    System.out.println(A.data);
 }
}
```

Recursion in Java

- Recursion in java is a process in which a **method calls itself continuously.**
- A method in java that calls itself is called recursive method.
- It makes the code compact but complex to understand.

Syntax:

```
returntype methodname(){
//code to be executed
methodname();//calling same method
}
```

Recursion in Java

- **Java Recursion Example 1: Infinite times**

```
public class RecursionExample1 {
static void p(){
System.out.println("hello");
p();
}

public static void main(String[] args) {
p();
}
}
```

Output:

```
hello
hello
...
java.lang.StackOverflowError
```

Recursion in Java

- **Java Recursion Example 2: Finite times**

```
public class RecursionExample2 {
static int count=0;
static void p(){
count++;
if(count<=5){
System.out.println("hello "+count);
p();
}
}
public static void main(String[] args) {
p();
}
}
```

Output:

```
hello 1
hello 2
hello 3
hello 4
hello 5
```

Recursion in Java

- **Java Recursion Example 3: Factorial Number**

```
public class RecursionExample3 {
   static int factorial(int n){
        if (n == 1)
          return 1;
        else
          return(n * factorial(n-1));
   }

public static void main(String[] args) {
System.out.println("Factorial of 5 is: "+factorial(5));
}
}
```

Output:
Factorial of 5 is: 120

Recursion in Java

- **Java Recursion Example 4: Fibonacci Series**

```
public class RecursionExample4 {
   static int n1=0,n2=1,n3=0;
    static void printFibo(int count){
      if(count>0){
          n3 = n1 + n2;
          n1 = n2;
          n2 = n3;
          System.out.print(" "+n3);
          printFibo(count-1);
       }
    }
```

```
public static void main(String[] args) {
    int count=15;
     System.out.print(n1+" "+n2);
         //printing 0 and 1
     printFibo(count-2);
     //n-2 because 2 numbers are already printed
}
}
```

Output:

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

More details: https://www.javatpoint.com/recursion-in-java

Java instance of Operator

- The **java instanceof operator** is used to test whether the object is an instance of the specified type (**class or subclass or interface**).
- The instanceof in java is also known as type *comparison operator* because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has null value, it returns false.

Simple example of java instanceof

Let's see the simple example of instance operator where it tests the current class.

```
class Simple1{
 public static void main(String args[])
{
 Simple1 s=new Simple1();
 System.out.println(s instanceof Simple1);  //true
 }
}
```

Output:
true

Java instance of Operator

- An object of subclass type is also a type of parent class. For example, if Dog extends Animal then object of Dog can be referred by either Dog or Animal class.

- Another example of java instanceof operator

```
class Animal{}
class Dog1 extends Animal{//Dog inherits Animal

 public static void main(String args[]){
 Dog1 d=new Dog1();
 System.out.println(d instanceof Animal);//true
 }
}
```

Output:
true

Java instance of Operator

- **instanceof in java with a variable that have null value**
- If we apply instanceof operator with a variable that have null value, it returns false. Let's see the example given below where we apply instanceof operator with the variable that have null value.

```
class Dog2{
 public static void main(String args[]){
  Dog2 d=null;
  System.out.println(d instanceof Dog2);  //false
 }
}
```

Output:
false

More details:

https://www.javatpoint.com/downcasting-with-instanceof-operator

Java enum Class

- The **Enum in Java** is a data type which **contains a fixed set of constants.**
- It can be used for days of the week (SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, and SATURDAY) , directions (NORTH, SOUTH, EAST, and WEST), season (SPRING, SUMMER, WINTER, and AUTUMN or FALL), colors (RED, YELLOW, BLUE, GREEN, WHITE, and BLACK) etc. According to the Java naming conventions, we should have all constants in capital letters. So, we have enum constants in capital letters.
- Java Enums can be thought of as classes which have a fixed set of constants (a variable that does not change).
- **The Java enum constants are static and final implicitly. It is available since JDK 1.5.**
- Enums are used to create our own data type like classes. The **enum** data type (also known as Enumerated Data Type) is used to define an enum in Java. Unlike C/C++, enum in Java is more *powerful*. Here, we can define an enum either inside the class or outside the class.

Java enum Class

- Java Enum internally inherits the *Enum class*, so it cannot inherit any other class, but it can implement many interfaces. We can have fields, constructors, methods, and main methods in Java enum.

Points to remember for Java Enum

- Enum improves type safety
- Enum can be easily used in switch
- Enum can be traversed
- Enum can have fields, constructors and methods
- Enum may implement many interfaces but cannot extend any class because it internally extends Enum class

Java enum Class

- **Simple Example of Java Enum**

```
class EnumExample1{
//defining the enum inside the class
public enum Season { WINTER, SPRING, SUMMER, FALL }
//main method
public static void main(String[] args) {
//traversing the enum
for (Season s : Season.values())
System.out.println(s);
}}
```

Output:

```
WINTER
SPRING
SUMMER
FALL
```

Java enum Class

- **Java Enum Example: Defined inside class**

```
class EnumExample3{
enum Season { WINTER, SPRING, SUMMER, FALL; }
        //semicolon(;) is optional here
public static void main(String[] args) {
Season s=Season.WINTER;//enum type is required to access WINTER
System.out.println(s);
}}
```

Output:
WINTER

More details:

https://www.javatpoint.com/enum-in-java